TECHNICAL COLLEGE OF THE LOWCOUNTRY

Computer Technology – Programming Concentration

# GymSync

A Hybrid Gym Management System with Offline Synchronization

**Final Capstone Report**

CPT 264 – Systems and Procedures


Student

**Keenan Speaks**

Instructor

**Orlando Macias Sanchez**


Date

**April 2026**

This project demonstrates the full application of the Systems Development Life Cycle (SDLC), integrating analysis, design, and AI-assisted prototyping into a unified system solution.

## Table of Contents

GymSync....1
Final Capstone Report....1
Table of Contents....2
0.0 Introduction....4
0.1 Project Overview....4
0.2 Purpose of the Capstone....4
0.3 Scope of the Project....4
0.4 Document Organization....4
1.0 System Vision & Problem Statement....6
1.1 System Name....6
1.2 Organization and User Context....6
1.3 Problem Statement....6
1.4 Proposed Solution Overview....7
1.5 Stakeholders and User Types....8
1.6 System Scope....8
2.0 Project Charter & Project Plan....10
2.1 Project Overview....10
2.2 Purpose of the System....10
2.3 Project Objectives....10
2.4 Project Scope Summary....11
In Scope....11
Out of Scope....11
2.5 Stakeholders and Roles....11
2.6 Risks, Assumptions, and Constraints....12
Risks....12
Assumptions....12
Constraints....13
2.7 High-Level Project Schedule....13
3.0 Requirements Specification....15
3.1 Requirements Overview....15
3.2 Functional Requirements....15
3.3 Non-Functional Requirements....16
3.4 Assumptions....16
3.5 Dependencies....17

3.6 Requirements Scope Validation ..... 17
4.0 Use Case Analysis ..... 18
4.1 Use Case Overview ..... 18
4.2 Actors Identification ..... 18
4.3 Use Case List ..... 19
4.4 Use Case Specifications ..... 19
UC-1: Member Check-In (Online/Offline) ..... 19
UC-2: Point-of-Sale (POS) Transaction (Online/Offline) ..... 20
UC-3: Schedule Class and Manage Roster ..... 21
UC-4: Create or Update Member Profile ..... 22
UC-5: Generate Reports (Administrative) ..... 23
4.5 Use Case Coverage Explanation ..... 23
5.0 Process Modeling & Workflow Analysis ..... 25
5.1 Process Modeling Overview ..... 25
5.2 Process Identification ..... 25
5.3 Process Diagrams ..... 26
Figure 5.1: Member Check-In Process (Online/Offline) ..... 26
Figure 5.2: Point-of-Sale Process (Online/Offline) ..... 27
Figure 5.3: Class Scheduling and Roster Management Process ..... 27
Figure 5.4: Offline Sync and Reconciliation Process ..... 28
5.4 Process Descriptions ..... 29
P-1: Member Check-In (Online/Offline) ..... 29
P-2: Point of Sale (Online/Offline) ..... 30
P-3: Class Scheduling, Roster, and Attendance ..... 30
P-4: Offline Sync and Reconciliation ..... 31
5.5 Process-to-Use Case Alignment ..... 31
6.0 Data Modeling & Data Dictionary ..... 33
6.1 Data Modeling Overview ..... 33
6.2 Entity Identification ..... 33
6.3 Attributes Overview ..... 34
6.4 Relationships ..... 34
6.5 Conceptual Data Model Diagram ..... 34
6.6 Data Dictionary (Summary) ..... 35
7.0 System Design Strategy & Transition Plan ..... 38
7.1 Design Transition Overview ..... 38
7.2 Design Goals ..... 38

7.3 System Design Scope ..... 39
7.4 Technology and Platform Assumptions ..... 39
7.5 Design Constraints and Risks ..... 40
8.0 System Architecture & Component Design ..... 42
8.1 Architecture Overview ..... 42
8.2 Architectural Style ..... 42
8.3 System Components ..... 43
8.4 Architecture Diagram ..... 43
8.5 Prototype Implementation Approach ..... 44
9.0 User Interface Design & Interaction Modeling ..... 46
9.1 UI Design Overview ..... 46
9.2 User Interface Design Principles ..... 46
9.3 Screen Identification ..... 47
9.4 Interface Structure Diagram ..... 47
9.5 UI Wireframes / Mockups ..... 48
9.6 Interaction Descriptions ..... 53
10.0 Program Logic & System Behavior Design ..... 55
10.1 Program Design Overview ..... 55
10.2 Feature Identification ..... 55
10.3 Program Logic Descriptions ..... 55
10.3.1 Login & Role-Based Access ..... 56
10.3.2 Member Check-In (Online/Offline) ..... 56
10.3.3 Point-of-Sale (POS) Transaction ..... 57
10.3.4 Class Enrollment & Capacity Management ..... 57
10.4 Error Handling & Validation Logic ..... 58
10.5 Offline Handling & System Behavior ..... 58
10.6 Backend Simulation Strategy ..... 59
11.0 Data Storage & State Management Design ..... 60
11.1 Data Storage Overview ..... 60
11.2 Data Storage Requirements ..... 60
11.3 Conceptual Storage Design ..... 61
11.4 Key Data Relationships (Expanded) ..... 61
11.5 Data Lifecycle & State Management ..... 62
11.6 State Management During User Interaction (Expanded) ..... 63
State Synchronization Behavior ..... 63
11.7 Prototype Data Simulation Strategy ..... 64

12.0 Product Requirements Document, Prototype Implementation & System Demonstration ..... 65
12.1 PRD Overview ..... 65
12.2 System Goals and Users ..... 65
System Goals ..... 65
Intended Users ..... 65
12.3 Functional Requirements Traceability ..... 66
12.4 Assumptions and Constraints ..... 66
Assumptions ..... 66
Constraints ..... 66
12.5 Prototype Implementation Using Windsurf ..... 66
Development Approach ..... 67
Technology Stack ..... 67
Core Features Implemented ..... 67
12.6 System Behavior & Workflow Execution ..... 67
Member Check-In ..... 67
Class Booking ..... 67
Point-of-Sale (POS) ..... 67
Offline Simulation ..... 68
12.7 Data Simulation Strategy ..... 68
12.8 PRD Instructions for AI Coding Assistant ..... 68
Implementation Guidelines ..... 68
Build Order ..... 69
AI Prompt Starter ..... 69
12.9 Testing and Validation ..... 69
Sample Test Cases ..... 69
12.10 System Demonstration & Deployment ..... 70
12.11 Live Demo and GitHub Links ..... 70
Figure 12.1 ..... 70
12.12 Implementation Summary ..... 71
13.0 Conclusion ..... 72
13.1 Project Summary ..... 72
13.2 Key Outcomes ..... 72
13.3 Real-World Application ..... 72
13.4 Closing ..... 72

# 0.0 Introduction

## 0.1 Project Overview

GymSync is a gym management system designed to support daily operations such as member check-ins, class scheduling, and point-of-sale (POS) transactions. The system addresses real-world operational challenges, including the need for fast front-desk workflows and continued functionality during internet outages.

## 0.2 Purpose of the Capstone

The purpose of this capstone project is to apply the Systems Development Life Cycle (SDLC) to design a complete system solution. This includes planning, analysis, design, and a prototype implementation supported by AI-assisted development tools.

This document consolidates all system artifacts into a unified design that demonstrates both technical understanding and practical application.

## 0.3 Scope of the Project

The GymSync system includes:

- Member management
- Check-in tracking
- Class scheduling and enrollment
- Point-of-sale (POS) functionality
- Offline data capture and synchronization

The project focuses on system design and a front-end prototype that simulates backend behavior using mock data.

## 0.4 Document Organization

This document is organized according to the SDLC phases, including system vision, requirements, modeling, design, and implementation planning.

Each section builds on the previous one to present a complete system design.

# 1.0 System Vision & Problem Statement

## 1.1 System Name

The system is named **GymSync Operations System ("GymSync")**, a hybrid gym management platform designed to support both online and offline operations.

## 1.2 Organization and User Context

GymSync is designed for a small to medium-sized fitness facility that offers memberships, personal training, and group classes. Daily operations are managed primarily by front desk attendants, with oversight from management.

Front desk staff are responsible for:

- Member check-ins
- Basic customer service
- Appointment scheduling
- Recording sales transactions

Management users are responsible for:

- Reviewing reports
- Approving adjustments
- Monitoring system activity and data accuracy

The system operates in a fast-paced environment where staff must process member interactions quickly and accurately. Any delays or errors directly impact customer experience and revenue.

To support real-world conditions, GymSync is designed to function in environments where internet connectivity may be unreliable or temporarily unavailable.

## 1.3 Problem Statement

The current system used by the gym relies heavily on continuous internet connectivity for all operations, including member check-ins, scheduling, and sales processing.

When internet connectivity is lost:

- Staff must manually record transactions using handwritten notes
- Data must later be re-entered into the system
- This results in increased workload, delays, and higher risk of human error
- The business may lose revenue due to missed or inaccurate transaction tracking

This dependency on constant connectivity creates operational inefficiencies and reduces overall reliability.

## 1.4 Proposed Solution Overview

GymSync is a hybrid, cloud-enabled gym management system designed to ensure continuous operation regardless of internet availability.

The system follows a dual infrastructure approach:

**Cloud-Based Core System**

- Hosts the primary database and business logic
- Provides centralized data storage for all locations
- Ensures high availability, security, and scalability
- Supports reporting, authentication, and system-wide data consistency

**Local Offline System (Edge Layer)**

- Runs on a local device or server within the gym
- Maintains a local database cache of critical data (members, products, schedules)
- Allows staff to continue operations during internet outages, including:
    - Member check-ins
    - Appointment scheduling
    - Sales transactions

**Synchronization Mechanism**

- When connectivity is restored, GymSync automatically:
    - Uploads locally stored transactions to the cloud database
    - Resolves conflicts using predefined rules (timestamp-based or manager review)
    - Updates the local database to match the cloud system

This dual-system architecture ensures:

- Business continuity during outages

- Data integrity across all locations
- Reduced manual effort and errors
- Scalability for multi-location operations

## 1.5 Stakeholders and User Types

| Role | Impact |
|---|---|
| **Members/End Users** | **Self-check-in, view account data, and ability to make self-purchases for faster service.** |
| **Front Desk Attendants** | **Perform member check-in, view member status, record sales, and reports.** |
| **Personal Trainers** | **View schedules, approved sessions, and follow-up.** |
| **Managers** | **Review, edit, and approve reports to ensure accuracy.** |
| **Sponsors/Owners** | **High-level reporting for business operations.** |
| **Facility Maintenance** | **Create and update maintenance tickets. Status update for competition.** |
| **IT Support/Administrators** | **Administrator level access: Manage user access, configuration, and data backups.** |

## 1.6 System Scope

**In Scope**

- Member check-in and status verification
- Member directory and search functionality
- Appointment scheduling and class management
- Point-of-sale (POS) transactions
- Offline data capture during outages

- Automatic data synchronization with cloud system
- Incident and maintenance reporting
- Role-based access control
- Manager dashboards and reporting

**Out of Scope**

- Direct banking or credit card processing without internet verification
- Full HR systems (payroll, employee performance tracking)
- Workout, nutrition, or medical advice features
- Automated physical access control systems (e.g., door entry systems)

•

# 2.0 Project Charter & Project Plan

## 2.1 Project Overview

**System Name:** GymSync Operations System ("GymSync")

GymSync is a centralized gym operations system designed for small to medium-sized fitness facilities that offer memberships, personal training, group classes, and retail products such as supplements and equipment.

The system supports core front-desk operations, including:

- Member check-in
- Appointment scheduling
- Point-of-sale (POS) transactions
- Incident and maintenance reporting
- Managerial reporting and oversight

GymSync follows a hybrid system architecture that combines a cloud-based core with a local offline system.

- The cloud system handles authentication, reporting, and centralized data storage
- The local system enables continued operation during internet outages

When connectivity is restored, GymSync synchronizes locally stored transactions with the cloud database and highlights any conflicts for resolution.

## 2.2 Purpose of the System

The purpose of GymSync is to eliminate operational disruptions caused by internet outages and manual data entry.

The system is designed to:

- Reduce front-desk delays
- Improve accuracy in member records and sales tracking
- Ensure continuous operation regardless of connectivity
- Strengthen security through role-based access control
- Provide managers with accurate, real-time reporting for decision-making

## 2.3 Project Objectives

GymSync will be considered successful when it meets the following objectives:

- Member check-ins and lookups are completed in under 15 seconds during normal operations
- Offline mode captures check-ins, appointments, and sales, and synchronizes data within 10 minutes of reconnection
- Role-based access control restricts and manages system permissions appropriately
- Data accuracy is maintained through conflict detection and resolution mechanisms
- Duplicate and conflicting records are minimized through synchronization rules and review processes

## 2.4 Project Scope Summary

### *In Scope*

- Member check-in and membership status verification
- Member directory search functionality
- Appointment scheduling for personal training and group sessions
- Point-of-sale (POS) transaction processing
- Incident and maintenance ticket reporting
- Offline data capture with synchronization and conflict resolution
- Role-based access control for all system users
- Audit logs for tracking system changes
- Dashboard and reporting features for management

### *Out of Scope*

- Direct banking or payment processor integration
- Full HR systems (payroll, employee performance tracking)
- Workout, nutrition, or medical advisory features
- Public-facing mobile applications

## 2.5 Stakeholders and Roles

**Project Sponsor / Owner**

- Approves system scope, priorities, and final acceptance

**Front Desk Staff (Primary Users)**

- Perform check-ins, manage members, schedule appointments, and record sales

**Personal Trainers**

- Manage schedules, confirm sessions, and track training activity

**Managers / Administrators**

- Approve changes, review reports, monitor system activity, and validate data

**Facility Maintenance Staff**

- Create, update, and close maintenance tickets

**IT Support / Administrators**

- Manage user access, system configuration, backups, and security

**System Analyst**

- Defines requirements, manages scope, and coordinates documentation

**UI/UX Designer**

- Designs system interface, layouts, and navigation

**AI-Assisted Developer**

- Builds and refines the prototype using approved tools

## 2.6 Risks, Assumptions, and Constraints

### *Risks*

- Offline synchronization complexity may lead to conflicts or duplicate records
- Security and privacy requirements may increase development complexity
- Time constraints may limit the depth of testing before final completion
- Additional features may introduce scope creep
- Learning curve for development tools

### *Assumptions*

- Staff and management are available for feedback during development
- The system designer has sufficient knowledge of SDLC and required tools
- Sample data will be used for testing and demonstration

### *Constraints*

- Database structures will use mock or sample data
- Development must align with the semester timeline
- No production database will be implemented
- No live deployment will occur during the project

## 2.7 High-Level Project Schedule

**Planning Phase (Weeks 1–2)**

- Project charter
- Business case
- Initial scope and feasibility

**Analysis Phase (Weeks 3–5)**

- Requirements gathering
- Use cases
- Process modeling
- Data requirements

**Design Phase (Weeks 6–8)**

- UI wireframes
- Navigation structure
- System architecture design
- Conceptual data model

**Implementation Phase (Weeks 9–12)**

- Interactive prototype
- Sample screens
- Validation rules
- Test cases
- Documentation updates

**Integration & Finalization (Weeks 13–14)**

- System revisions
- Risk review
- Final capstone report
- Presentation preparation

# 3.0 Requirements Specification

## 3.1 Requirements Overview

This section defines the functional and non-functional requirements for GymSync.

These requirements build upon the system vision (Section 1.0) and project charter (Section 2.0) by translating system scope into clear, testable specifications.

GymSync supports day-to-day gym operations across one or multiple locations, including member management, check-ins, class scheduling, and point-of-sale (POS) transactions. These requirements establish the baseline for system design, development, and evaluation.

## 3.2 Functional Requirements

The following functional requirements (FR-#) define the core system behavior:

**FR-1:** GymSync will allow staff to sign in and sign out using a username and password. The system will lock the account for 15 minutes after 5 failed login attempts.

**FR-2:** GymSync will allow authorized staff to create, read, update, and delete (CRUD) member profiles, including contact information and membership status.

**FR-3:** GymSync will record a member check-in with the date, time, staff member, and location when a QR code/key fob is scanned, or a member is manually selected.

**FR-4:** When the network is unavailable, GymSync will accept check-ins using cached member data and will synchronize queued check-ins within 10 minutes after connectivity is restored.

**FR-5:** GymSync will display a check-in confirmation screen showing the member's name, photo, membership status, and any active alerts.

**FR-6:** GymSync will allow staff to create and manage class schedules, enroll members, manage waitlists, and record attendance.

**FR-7:** GymSync will provide a point-of-sale (POS) system that allows staff to process sales, calculate taxes based on location, and generate receipts.

**FR-8:** GymSync will provide a daily dashboard displaying total check-ins, total sales, and upcoming classes, with an option to export data to CSV.

**FR-9:** GymSync will enforce role-based access control and restrict access to sensitive data such as financial information.

**FR-10:** GymSync will maintain an audit log of critical system actions, including profile changes, refunds, class cancellations, and role updates.

### 3.3 Non-Functional Requirements

The following non-functional requirements (NFR-#) define system quality and performance:

**NFR-1 (Usability):** Staff will complete a member check-in in three clicks or fewer from the home screen.

**NFR-2 (Accessibility):** The user interface will meet WCAG 2.1 Level AA standards for accessibility.

**NFR-3 (Performance):** The system will load the check-in screen and return member search results within 5 seconds on a standard broadband connection.

**NFR-4 (Reliability):** The system will remain available except during scheduled maintenance announced at least 48 hours in advance.

**NFR-5 (Compatibility):** The system will support modern browsers (Chrome, Firefox, Edge) on Windows and macOS and will be responsive on tablets and smartphones.

**NFR-6 (Security):** The system will enforce least-privilege access, encrypt data in transit (HTTPS/TLS), and securely store passwords using hashing.

**NFR-7 (Privacy):** The system will mask sensitive user information, including financial data.

**NFR-8 (Maintainability):** The system will use modular, reusable components to allow efficient updates and maintenance.

### 3.4 Assumptions

- Staff will have individual user accounts
- Gym locations will have at least one workstation and mobile-capable devices
- QR code or barcode scanning will be available
- Payment processing will be handled by a third-party provider
- Email and SMS notifications will rely on external services
- Staff training time will be limited (30–60 minutes)

### 3.5 Dependencies

- A central database/API to receive synchronized data
- Configured role-based access control system
- Location-based tax configuration
- Network and firewall access to required external services

### 3.6 Requirements Scope Validation

The requirements focus on GymSync's core operations:

- Member management
- Check-ins
- Class scheduling
- Basic point-of-sale (POS) functionality
- Offline data capture and synchronization

The system intentionally excludes:

- Payroll and HR systems
- Full accounting functionality
- Advanced forecasting
- Public mobile applications
- Extensive marketing features

The final system will deliver a fully integrated SDLC-based design, including documentation, system models, and a working prototype that demonstrates core workflows.

# 4.0 Use Case Analysis

## 4.1 Use Case Overview

This section defines how users (actors) interact with GymSync to complete system tasks.

Use cases translate system requirements into step-by-step user interactions and system responses. They serve as the foundation for process modeling, interface design, and system validation.

GymSync use cases support core operations:

- Member management
- Check-ins
- Class scheduling
- Point-of-sale (POS) transactions

They also incorporate key operational constraints, including **offline functionality and later synchronization with the cloud system**.

## 4.2 Actors Identification

**A1: Gym Member**

- Role: Customer who checks in, joins classes, and makes purchases
- Relationship: Initiates check-ins and class participation

**A2: Front Desk Staff**

- Role: Primary system user responsible for daily operations
- Relationship: Performs check-ins, sales, and member updates in both online and offline modes

**A3: Trainer / Class Instructor**

- Role: Manages class sessions and attendance
- Relationship: Uses schedules and rosters to manage sessions

**A4: Gym Manager / Administrator**

- Role: Oversees operations, reporting, and system configuration
- Relationship: Approves overrides, reviews reports, monitors synchronization

**A5: Payment Processor (External System)**

- Role: Third-party service for payment authorization
- Relationship: Processes and returns payment results

**A6: Local Devices**

- Role: Scanner, receipt printer, and local database
- Relationship: Supports offline operation and data capture

## 4.3 Use Case List

- **UC-1:** Member Check-In (Online/Offline)
- **UC-2:** Sell Item or Membership (POS) (Online/Offline)
- **UC-3:** Schedule Class and Manage Roster
- **UC-4:** Create or Update Member Profile
- **UC-5:** Generate Reports (Administrative Use Case)

## 4.4 Use Case Specifications

### UC-1: Member Check-In (Online/Offline)

**Primary Actor:** Front Desk Staff

**Description:**
Staff check members into the gym by scanning or searching. The system validates membership status and records the visit. Offline operation is supported with later synchronization.

**Preconditions:**

- Staff is authenticated
- Member exists or can be created

**Postconditions:**

- Check-in is recorded (cloud or local)
- Visit history is updated
- Offline check-ins are marked “Pending Sync”

**Main Success Flow:**

1. Staff selects “Check-In”

2. Staff scans QR/barcode or searches member
3. System retrieves profile and status
4. System displays eligibility
5. Staff confirms check-in
6. System records timestamp and workstation
7. System displays confirmation

**Alternate Flows:**

*Membership Expired*

- System prompts renewal or day pass → routes to UC-2

*Member Not Found*

- System suggests retrying or creating a new member → UC-4

*Offline Mode*

- System uses local database
- Stores check-in as "Pending Sync"
- Sync occurs when connection is restored

### UC-2: Point-of-Sale (POS) Transaction (Online/Offline)

**Primary Actor:** Front Desk Staff

**Description:**
Staff process sales for memberships and products. Transactions are recorded and synchronized if offline.

**Preconditions:**

- Staff is authenticated
- Product catalog is available

**Postconditions:**

- Sale is recorded
- Payment status is logged
- Offline sales marked "Pending Sync"

**Main Success Flow:**

1. Staff opens POS
2. Selects member or guest

3. Adds items to cart
4. System calculates total
5. Staff selects payment method
6. Payment is processed
7. System records sale and generates receipt

**Alternate Flows:**

*Card Declined*

- Retry or alternate payment

*Offline Mode*

- Card payments disabled
- Sale stored locally as "Pending Sync"

*Price Override*

- Manager authorization required
- Action logged in audit system

### UC-3: Schedule Class and Manage Roster

**Primary Actor:** Trainer

**Description:**
Trainer creates and manages class sessions and attendance.

**Preconditions:**

- User is authenticated
- Class rules are defined

**Postconditions:**

- Class session is saved
- Roster updated
- Attendance recorded

**Main Success Flow:**

1. Trainer opens schedule
2. Creates or edits session
3. Enters class details
4. System validates and saves

5. Session is published
6. Trainer manages roster
7. Attendance is recorded

**Alternate Flows:**

*Class Full*

- System offers waitlist

*Member Ineligible*

- System blocks enrollment

*Session Cancelled*

- System logs cancellation and notifies members

### UC-4: Create or Update Member Profile

**Primary Actor:** Front Desk Staff

**Description:**
Staff create or update member records. Offline entries sync later.

**Preconditions:**

- Staff is authenticated

**Postconditions:**

- Profile is saved
- Offline entries marked “Pending Sync”

**Main Success Flow:**

1. Staff selects member or creates new
2. System displays form
3. Staff enters information
4. System validates data
5. Profile is saved
6. Confirmation displayed

**Alternate Flows:**

*Duplicate Detected*

- System prompts merge or override

*Offline Mode*

- Profile stored locally
- Synced later

### UC-5: Generate Reports (Administrative)

**Primary Actor:** Manager

**Description:**
Managers generate reports for attendance, sales, and system activity.

**Preconditions:**

- Manager is authenticated

**Postconditions:**

- Report is displayed or exported

**Main Success Flow:**

1. Manager selects report type
2. Selects date range
3. System retrieves data
4. Report is displayed
5. Manager exports report if needed

**Alternate Flow:**

*Offline Mode*

- System displays locally available data
- Marks report as partial if needed

## 4.5 Use Case Coverage Explanation

These use cases represent the complete operational behavior of GymSync.

- UC-1 supports member check-ins and attendance tracking
- UC-2 supports sales and membership transactions
- UC-3 supports class scheduling and roster management
- UC-4 supports member profile management

- UC-5 supports managerial reporting and oversight

Together, these use cases translate system requirements into structured workflows, including handling for:

- Membership issues
- Payment failures
- Capacity limits
- Offline operation and synchronization

This ensures that GymSync's behavior is fully defined for process modeling, interface design, and system validation.

# 5.0 Process Modeling & Workflow Analysis

## 5.1 Process Modeling Overview

Process modeling describes how work is performed within GymSync by illustrating the sequence of activities, decision points, and interactions between users and the system.

These models translate use cases into structured workflows that:

- Guide user interface design
- Define system logic
- Validate operational behavior

GymSync process models include both front-end user actions and conceptual backend processes such as:

- Local data storage
- Cloud synchronization
- Payment authorization
- Offline queue management

This ensures that both online and offline operations are fully represented.

## 5.2 Process Identification

**P-1: Member Check-In (Online/Offline)**

- Records member visits through scan or search
- Validates membership status
- Stores check-in in cloud or local queue

**P-2: Point of Sale (Online/Offline)**

- Processes product and membership sales
- Calculates totals and taxes
- Handles payments and generates receipts
- Queues offline transactions for synchronization

**P-3: Class Scheduling, Roster, and Attendance**

- Creates and manages class sessions
- Publishes schedules and manages rosters
- Enforces capacity and eligibility rules
- Records attendance

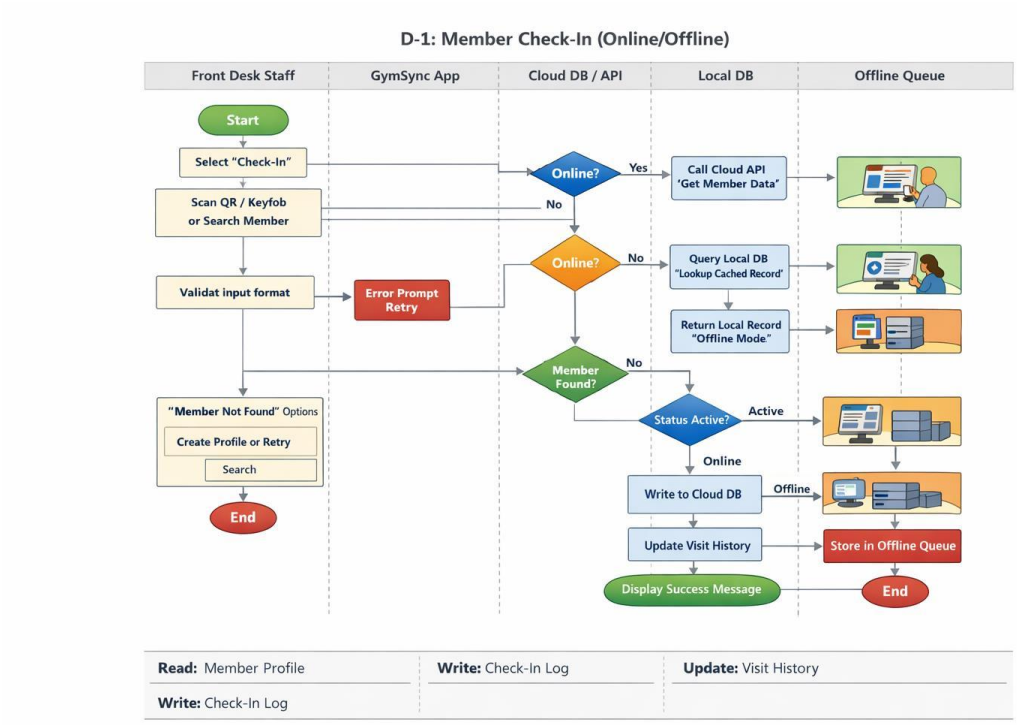
**P-4: Offline Sync and Reconciliation**

- Uploads queued offline data to cloud
- Detects and resolves conflicts
- Updates local data to match cloud records

## 5.3 Process Diagrams

The following diagrams visually represent GymSync workflows, decision logic, and data movement between system components.

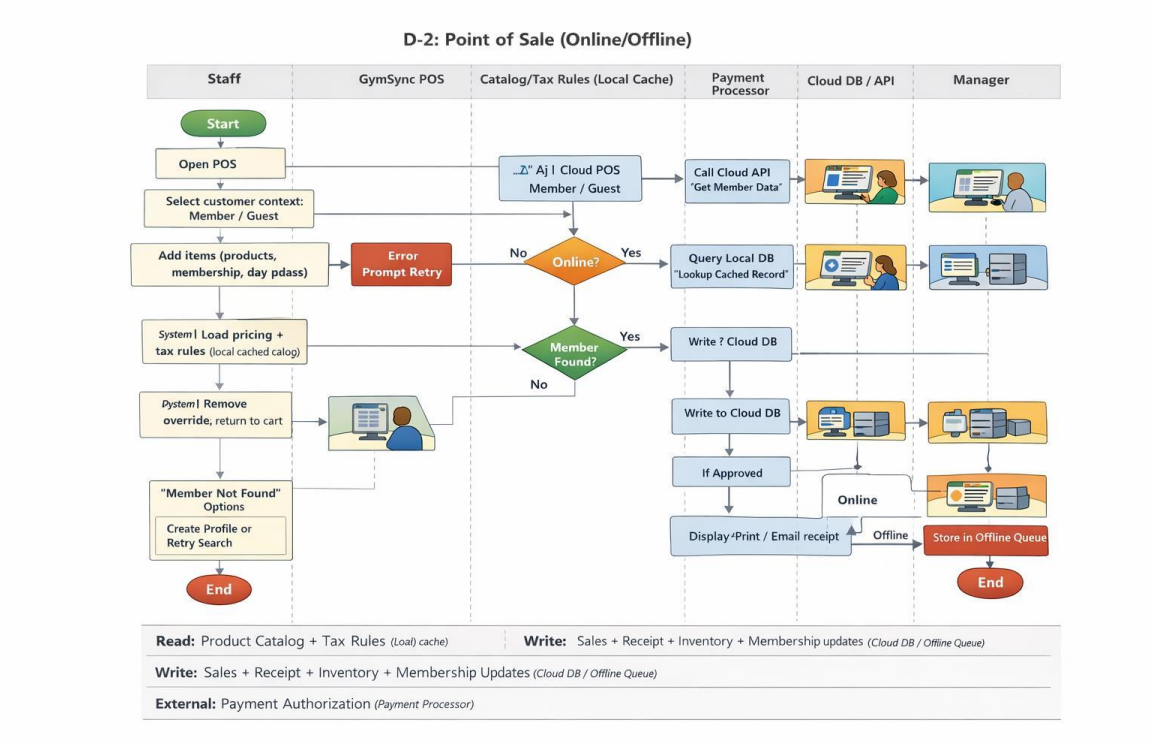
*Figure 5.1: Member Check-In Process (Online/Offline)*




***Description:***
This diagram illustrates the check-in workflow, including online vs offline branching,

member validation, and recording check-in data to either the cloud database or local offline queue.

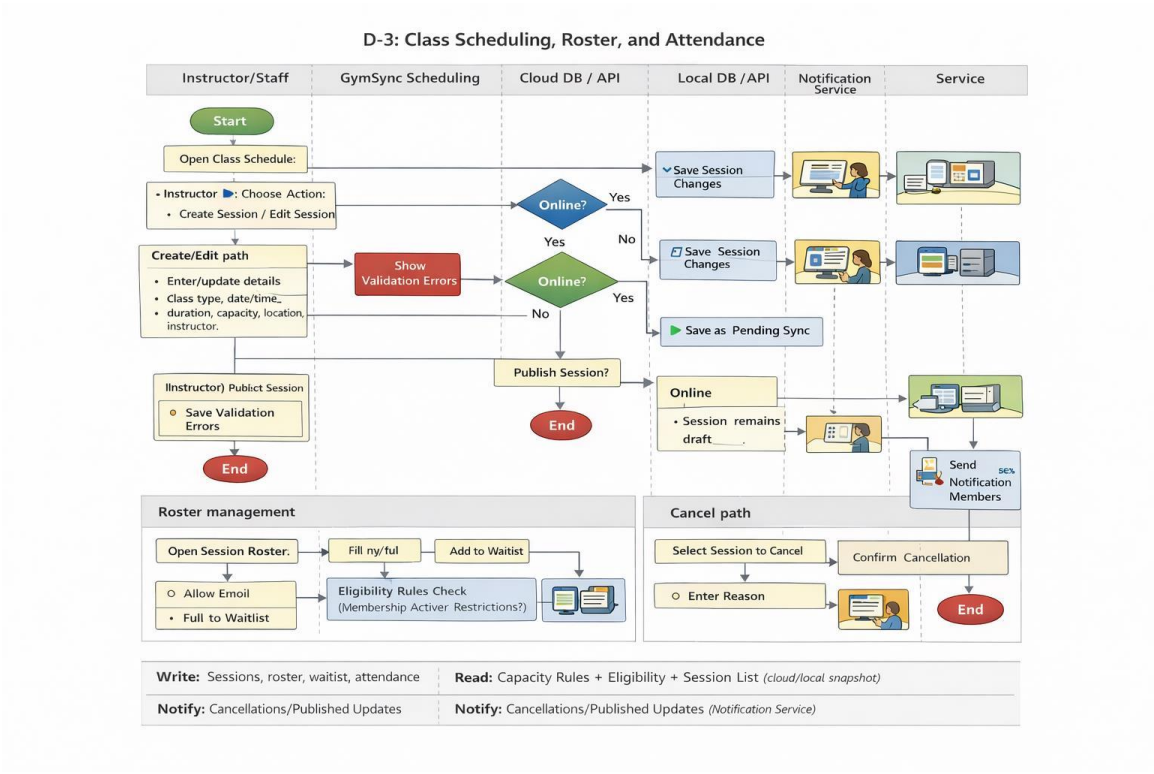
### *Figure 5.2: Point-of-Sale Process (Online/Offline)*




***Description:***

This diagram shows the POS workflow, including item selection, tax calculation, payment processing, and offline transaction handling with “Pending Sync” status.

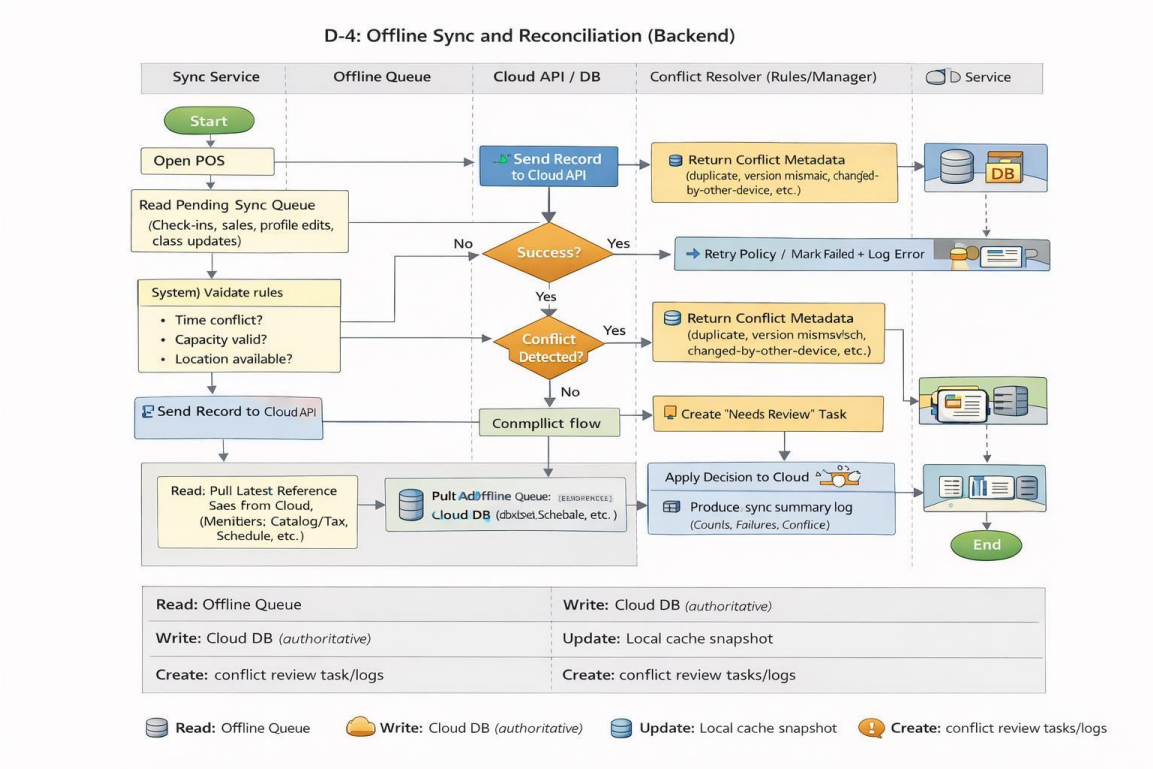
*Figure 5.3: Class Scheduling and Roster Management Process*




***Description:***

This diagram represents class creation, scheduling, roster management, capacity validation, and attendance tracking.

*Figure 5.4: Offline Sync and Reconciliation Process*




*Description:*
This diagram illustrates backend synchronization, including offline queue processing, conflict detection, resolution logic, and updating local and cloud databases.

## 5.4 Process Descriptions

### *P-1: Member Check-In (Online/Offline)*

This process captures a member visit from check-in initiation through confirmation.

Staff scan a QR code/key fob or search for a member. The system determines connectivity status:

- **Online:** Retrieves member data from the cloud
- **Offline:** Retrieves data from the local database and displays offline status

GymSync:

- Records check-in timestamp and workstation
- Updates visit history

- Displays confirmation

If membership is invalid:

- System offers renewal or day pass options (routes to POS)

If member is not found:

- System offers profile creation or manual entry

### *P-2: Point of Sale (Online/Offline)*

This process completes sales transactions for membership and retail items.

Staff:

- Select member or guest
- Add items to cart
- System calculates subtotal, tax, and total

Additional logic:

- Price overrides require manager authorization
- Approval and reason are logged

Payment handling:

- Card payments require online authorization
- Cash/check accepted in all modes

If offline:

- Card payments are unavailable
- Transactions are stored as “Pending Sync”

Approved transactions:

- Update sales records
- Update memberships and inventory
- Generate receipt

### *P-3: Class Scheduling, Roster, and Attendance*

This process allows instructors to manage class sessions.

Instructors:

- Create or edit sessions
- Define class details (type, time, capacity, location)

System:

- Validates scheduling rules
- Publishes sessions
- Manages roster and waitlist

During/after class:

- Attendance is recorded
- Data is stored for reporting

If cancelled:

- System logs cancellation
- Notifications may be sent

### *P-4: Offline Sync and Reconciliation*

This backend process ensures data consistency between local and cloud systems.

When connectivity is restored:

- Sync service scans the offline queue
- Uploads transactions (check-ins, sales, profiles)

System performs:

- Conflict detection (duplicates, mismatched updates)
- Automatic resolution using predefined rules
- Manual escalation for unresolved conflicts

After resolution:

- Records are marked as synced
- Local database is updated to match cloud data

## 5.5 Process-to-Use Case Alignment

Each process aligns directly with defined use cases:

- **UC-1: Member Check-In → P-1**
- **UC-2: POS Transactions → P-2**
- **UC-3: Class Scheduling → P-3**
- **UC-4: Member Management → P-1 & P-4**
- **Offline Behavior Across All Use Cases → P-4**

•

# 6.0 Data Modeling & Data Dictionary

## 6.1 Data Modeling Overview

Data modeling identifies the information GymSync must store, process, and present.

The conceptual data model focuses on:

- Core business entities
- Key attributes
- Relationships between entities

This model supports:

- Membership management
- Check-ins
- Class scheduling and enrollment
- Point-of-sale transactions
- Notifications
- Offline-to-online synchronization

Conceptual modeling is independent of database implementation and focuses only on structure and relationships.

## 6.2 Entity Identification

The following entities define the core structure of GymSync:

- **Member** – Represents a gym customer
- **MembershipPlan** – Defines membership tiers and benefits
- **CheckIn** – Records member visits
- **ClassSession** – Represents scheduled classes
- **Enrollment** – Links members to class sessions (resolves M:N relationship)
- **StaffUser** – Represents employees and system users
- **Sale** – Represents transactions
- **SaleLineItem** – Represents items within a sale
- **Product** – Represents goods or services sold
- **PaymentTransaction** – Represents payments applied to sales
- **Notification** – Represents system messages to members
- **SyncEvent** – Tracks offline-to-online synchronization activity

### 6.3 Attributes Overview

Each entity includes meaningful conceptual attributes such as:

- **Member** → Name, Contact Info, Membership Status
- **Sale** → Date, Total, Status
- **Product** → Name, Price, Category
- **PaymentTransaction** → Method, Amount, Status
- **SyncEvent** → Event Type, Status, Conflict Flag

Attributes are defined at a conceptual level and intentionally avoid physical database implementation of details.

### 6.4 Relationships

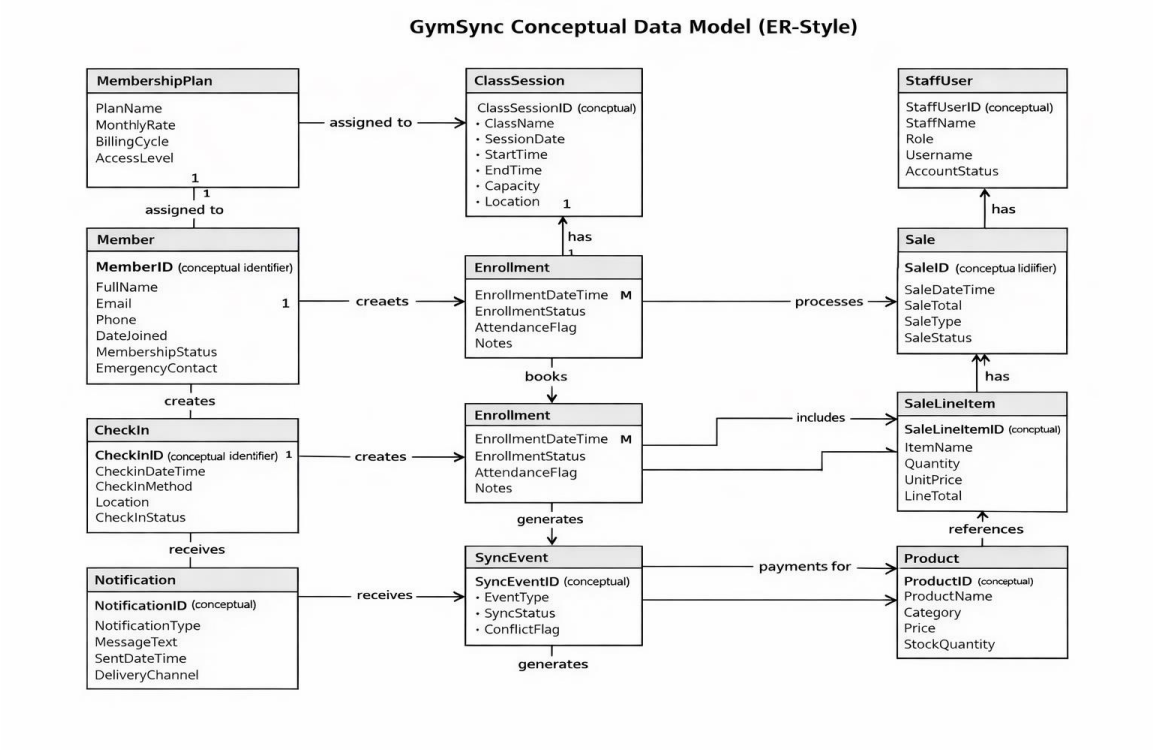
Key relationships include:

- **Member** → CheckIn (1:M)
- **MembershipPlan** → Member (1:M)
- **Member** ↔ ClassSession (M:N resolved by Enrollment)
- **StaffUser** → Sale (1:M)
- **Sale** → SaleLineItem (1:M)
- **Product** → SaleLineItem (1:M)
- **Sale** → PaymentTransaction (1:M)
- **Member** → Notification (1:M)
- **Offline actions** → SyncEvent (1:M)

These relationships ensure accurate representation of system behavior and data flow.

### 6.5 Conceptual Data Model Diagram

**Figure 6.1: GymSync Conceptual Data Model (ER Diagram)**




## 6.6 Data Dictionary (Summary)

**Member**
Purpose: Represents a gym customer
Key Attributes: Name, Email, Phone, Membership Status

**MembershipPlan**
Purpose: Defines membership tiers and pricing
Key Attributes: Plan Name, Rate, Billing Cycle

**CheckIn**
Purpose: Records member visits
Key Attributes: DateTime, Method, Status

**ClassSession**
Purpose: Represents scheduled classes
Key Attributes: Class Name, Date, Time, Capacity

**Enrollment**
Purpose: Links members to class sessions
Key Attributes: Enrollment Date, Status, Attendance

**StaffUser**
Purpose: Represents system users and employees
Key Attributes: Name, Role, Account Status

**Sale**
Purpose: Represents transactions
Key Attributes: DateTime, Total, Status

**SaleLineItem**
Purpose: Represents individual items in a sale
Key Attributes: Item Name, Quantity, Price

**Product**
Purpose: Represents items for sale
Key Attributes: Name, Category, Price

**PaymentTransaction**
Purpose: Represents payments for sales
Key Attributes: Method, Amount, Status

**Notification**
Purpose: Represents system messages
Key Attributes: Type, Message, Delivery Channel

**SyncEvent**
Purpose: Tracks offline-to-online synchronization
Key Attributes: Event Type, Status, Conflict Flag

# 7.0 System Design Strategy & Transition Plan

## 7.1 Design Transition Overview

This section explains the transition from system analysis to system design for GymSync.

During the analysis phase, the project established what GymSync must do through requirements, use cases, process models, and the conceptual data model. The design phase builds those analysis artifacts by defining how GymSync will be structured to support those needs.

The analysis artifacts informed system design in the following ways:

- Requirements defined the core features of the system, including membership management, check-ins, point-of-sale transactions, class scheduling, and offline synchronization.
- Use cases defined user goals and expected system behavior.
- Process models defined workflow steps, validations, exceptions, and synchronization behavior.
- The conceptual data model defined the entities, attributes, and relationships the system must support.

GymSync is designed as a complete real-world system conceptually, including backend services, data handling, and integrations. However, the prototype for this project will focus on a front-end implementation supported by mock data and simulated backend behavior.

## 7.2 Design Goals

**Usability and Speed for Front-Desk Workflows**
Gym staff must complete check-ins and sales quickly during peak operating hours. The interface must be simple, efficient, and easy to navigate.

**Consistency Across Screens and Roles**
GymSync supports multiple user roles, including front desk staff, trainers, managers, and administrators. Consistent layout, labeling, and navigation will reduce training time and improve usability.

**Reliability in Online and Offline Conditions**
Gym operations must continue even when internet access is unavailable. The design

must support local data capture, queued actions, and synchronization when connectivity returns.

**Scalability for Growth**
Although the prototype uses mock data, the system design should support multiple locations, larger member volumes, and increased transaction activity.

**Maintainability Through Modular Design**
GymSync should be organized into clear modules such as Members, Check-In, Classes, POS, Notifications, and Reporting to simplify future updates and maintenance.

## 7.3 System Design Scope

**Components Fully Designed Conceptually**

- User interface design, including screen layouts, navigation structure, and role-based views
- Backend services for membership management, check-ins, point-of-sale, inventory, scheduling, and notifications
- Data handling and storage, including conceptual schema, relationships, and validation rules
- Offline synchronization processes, including queue handling, conflict resolution, and audit logging

**Components Implemented in the Prototype**

- Member search and profile view
- Check-in workflow
- Class schedule browsing and booking flow
- POS checkout screens
- Basic administrative screens for membership plans and products

**Features Simulated with Mock Data**

- Member profiles
- Membership plans
- Check-in history
- Product catalog
- Class sessions
- Sales records
- Sync status indicators
- Payment approval messages
- Notification displays

## 7.4 Technology and Platform Assumptions

**Conceptual Backend Technologies**

- RESTful API for communication between front-end and backend services
- Relational database for centralized data storage
- Role-based authentication and access control
- Local storage with queued synchronization for offline activity
- Logging and audit tracking for operational events

**Front-End Technologies**

- HTML, CSS, and JavaScript for screen design and interactions
- Reusable UI components for maintainability
- Mock data using JSON or in-memory objects to simulate system responses

**Primary AI Development Tool**
The primary AI coding tool selected for the GymSync prototype is **Windsurf**. Windsurf will be used as the main development environment for front-end prototyping, interface generation, workflow simulation, and iterative refinement of the user experience.

## 7.5 Design Constraints and Risks

**Design Constraints**

- The prototype is limited to front-end simulation rather than full backend implementation
- Data interactions must be demonstrated using mock or static data
- The project must remain focused on core system workflows
- Development must fit within the semester timeline

**Potential Risks**

- Scope creep may make the prototype too large to complete effectively
- Offline synchronization logic may become too complex if overdesigned
- New tools or frameworks introduced too late may slow progress

**Mitigation Strategies**

- Prioritize core workflows such as check-in, POS, class booking, and membership management
- Keep synchronization behavior conceptually clear and visually represented
- Use modular screen design and reusable interface components
- Limit the prototype to core workflows and key exceptions

•

# 8.0 System Architecture & Component Design

## 8.1 Architecture Overview

System architecture defines how GymSync's components interact to support system functionality, performance, and reliability.

The architecture provides a high-level blueprint showing how the following features work together to support gym operations:

- User interfaces
- Backend services
- Data storage
- External systems

GymSync's architecture is designed to:

- Support core operations such as check-in, POS transactions, class scheduling, and reporting
- Maintain secure handling of sensitive data such as payments
- Ensure continued operation during internet outages through offline capability and synchronization

The architecture translates system requirements and process models into a structured system design.

## 8.2 Architectural Style

GymSync uses an **n-tier client-server architecture with a layered design**.

- **Client Layer:** Handles presentation logic through web or mobile interfaces
- **Application Layer (Conceptual):** Handles business logic, validation, and workflow processing
- **Data Layer (Conceptual):** Handles storage, retrieval, and synchronization of data

To support offline-first operation, GymSync includes an **on-premise edge node** that:

- Stores data locally during outages
- Queues transactions for synchronization
- Reconciles data with the cloud when connectivity returns

This architecture supports scalability, maintainability, and reliability across multiple locations.

## 8.3 System Components

**User Interface (Front End)**

- Member-facing screens, POS interface, and admin portal
- Displays data, collects input, and performs basic validation
- Manages offline indicators and queued actions

**Backend Services (Conceptual)**

- API routing and request handling
- Authentication and role-based access control
- Core business logic (members, check-ins, POS, classes)
- Synchronization and conflict resolution
- Notifications and reporting services

**Data Storage Layer (Conceptual)**

- Cloud database (system of record)
- Local database for offline operation
- Cache and message queue
- Sync ledger for tracking offline transactions

**External Systems**

- Payment processor (secure, tokenized transactions)
- Email/SMS notification services
- Optional integrations (door access, accounting systems)

## 8.4 Architecture Diagram

**Figure 8.1: GymSync System Architecture Diagram**

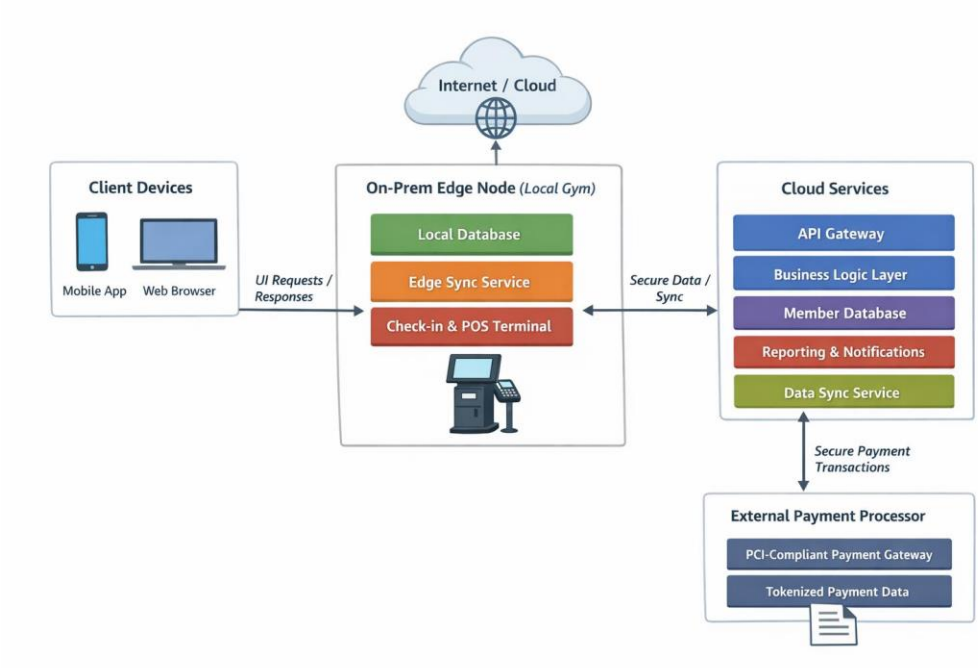



**Diagram Description:**
The architecture diagram illustrates how GymSync components interact across environments.

As shown in the original diagram:

- Client devices (mobile and web) communicate with the system through the internet
- The on-premise edge node handles local operations, including:
  - Local database storage
  - Check-in and POS processing
  - Edge synchronization services
- Cloud services provide:
  - API gateway
  - Business logic processing
  - Centralized data storage
  - Reporting and notifications
- External payment processors handle secure payment authorization and processing

The diagram clearly represents the separation between local operations and cloud services, as well as secure data flow between components.

## 8.5 Prototype Implementation Approach

The GymSync prototype will be developed using Windsurf as the primary AI-assisted

development tool.

**Front-End Implementation**

- Member interface screens
- Front desk POS interface
- Administrative screens
- Navigation, forms, and validation logic

**Simulated Backend Behavior**

- API interactions represented using mock endpoints
- Data operations simulated with static datasets (e.g., JSON)
- System responses mimic real-world backend behavior

**Offline Behavior Simulation**

- Local storage used to queue actions such as check-ins, sales, and bookings
- UI indicators display system status (Online / Offline / Pending Sync)
- When switching to online mode, queued actions are processed and results are displayed

**External Integration Simulation**

- Payment processing simulated with approval/decline messages
- Notifications displayed through UI alerts or message panels
- No real external transactions are executed

•

# 9.0 User Interface Design & Interaction Modeling

## 9.1 UI Design Overview

User Interface (UI) design defines how users interact with GymSync and how the system presents information and actions.

A well-designed interface improves usability by:

- Reducing the number of steps required to complete tasks
- Minimizing user errors
- Increasing efficiency during high-volume operations

The GymSync interface is directly derived from system use cases and process models, including:

- Member check-in workflow
- Point-of-sale (POS) workflow
- Class scheduling and enrollment
- Administrative reporting and system management

In the original wireframes, each screen is designed to reflect real-world workflows and decision points.

## 9.2 User Interface Design Principles

**Consistency**
Navigation structure, labels, and layout patterns remain consistent across all screens, including a left-side navigation menu, header bar, and status footer.

**Simplicity and Efficiency**
Common actions such as check-in, member lookup, and sales processing are accessible within one or two interactions. Default values and selection controls reduce manual input.

**Visibility of System Status**
A persistent status bar displays:

- Online / Offline mode
- Pending Sync indicators
- System alerts

This ensures users are always aware of system state.

**Accessibility**
The interface supports:

- Readable font sizes
- Clear field labels
- High-contrast sections
- Touch-friendly buttons

This allows use across different devices and user skill levels.

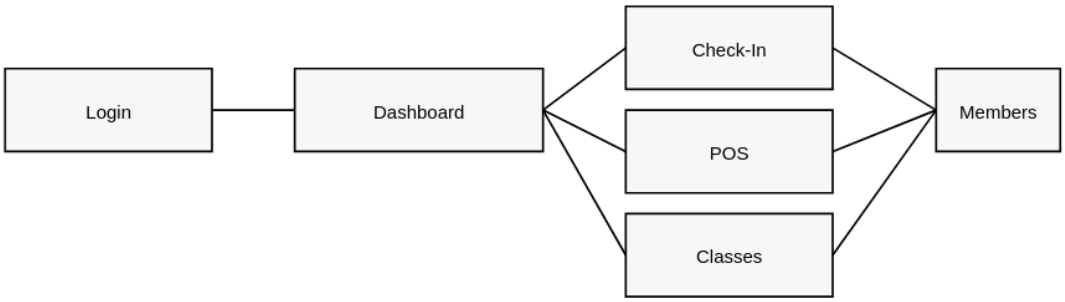
## 9.3 Screen Identification

| Screen Name | Primary Users | Purpose |
|---|---|---|
| Dashboard / Home | Staff, Manager/Admin | Displays alerts, summaries, and quick actions |
| Member Check-In | Staff | Find member, verify status, record visits |
| Point-of-Sale (POS) | Staff | Process sales and payments |
| Classes & Enrollment | Staff, Trainer | Manage schedules and enrollments |
| Reports & Admin | Manager/Admin | Generate reports and manage system data |

Each screen directly supports a core system workflow.

## 9.4 Interface Structure Diagram

**Figure 9.1: Interface Navigation Flow**




*Interface Structure Diagram (high-level navigation flow)*

**Diagram Description:**
The interface structure diagram illustrates how users move between system screens.

As shown in the diagram:

- Users begin at the Login screen
- Navigate to the Dashboard
- From the Dashboard, users access:
    - Check-In
    - POS
    - Classes
    - Members

This structure supports quick access to core workflows.

## 9.5 UI Wireframes / Mockups

**Figure 9.2: Login Screen**

**Figure 9.3: Dashboard / Home Screen**

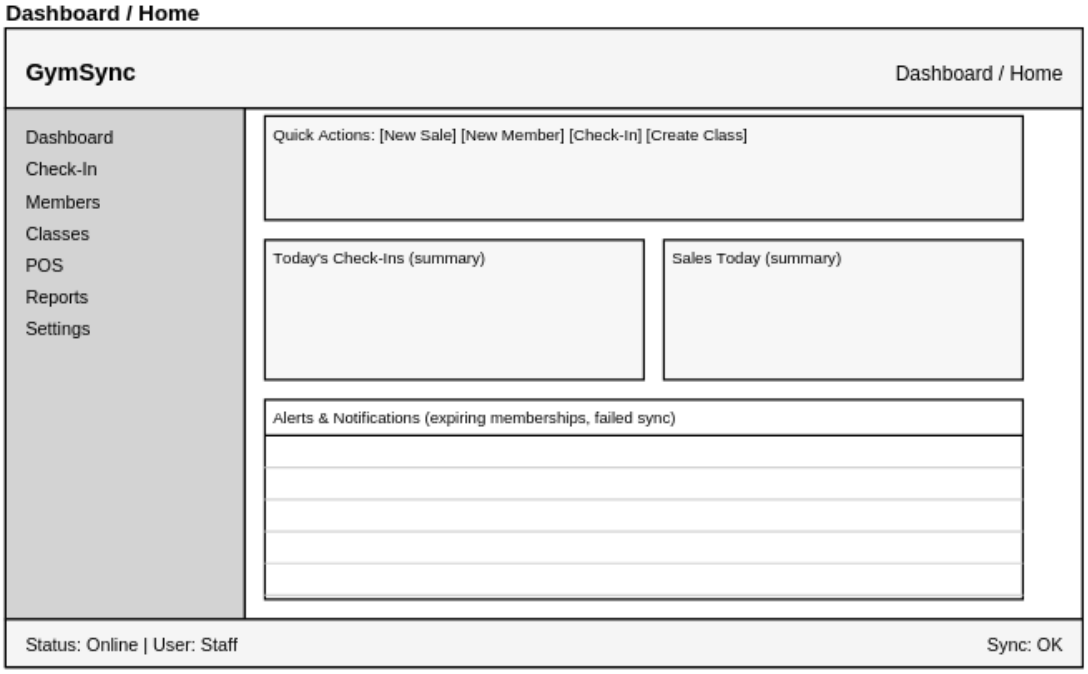



**Figure 9.4: Member Check-In Screen**

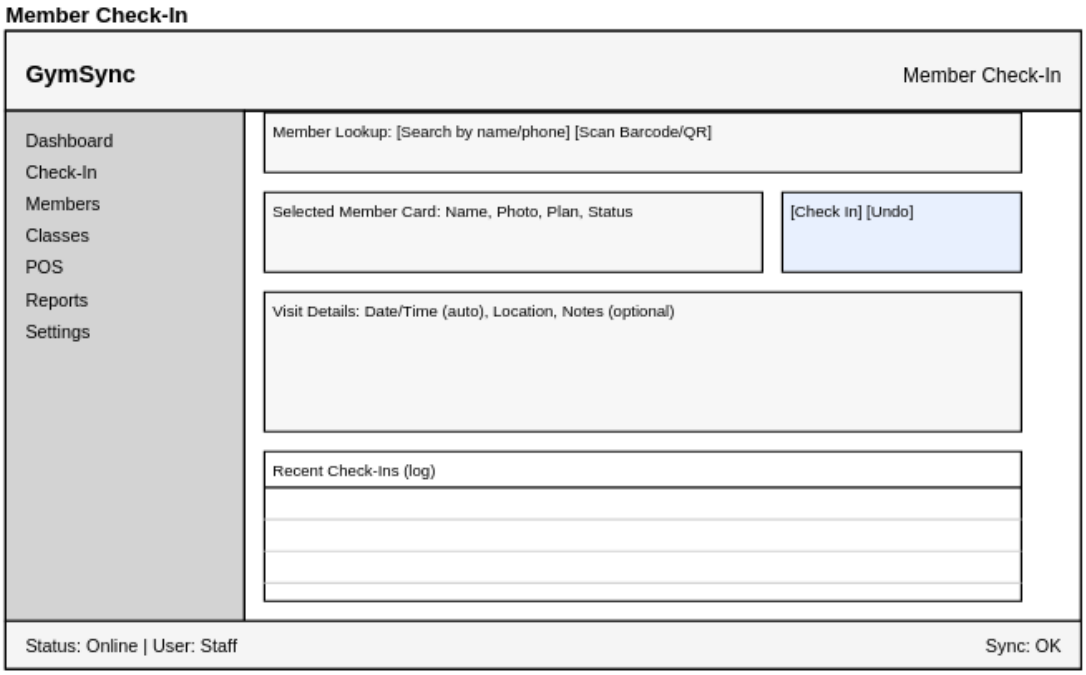


**Figure 9.5: Point-of-Sale (POS) Screen**

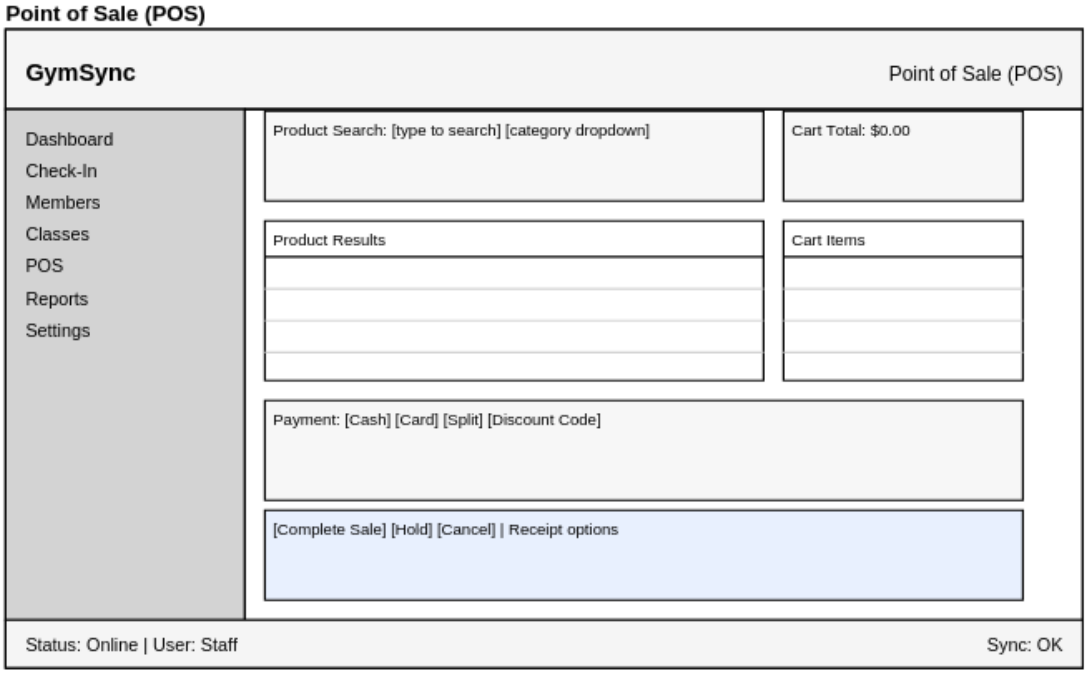



**Figure 9.6: Classes & Enrollment Screen**

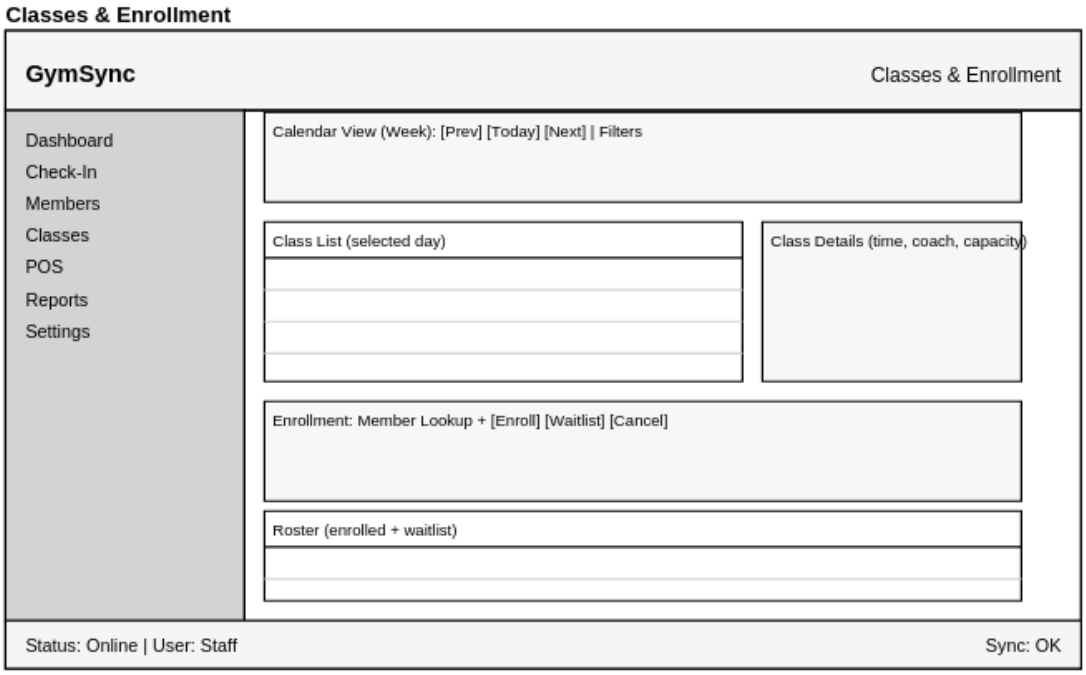
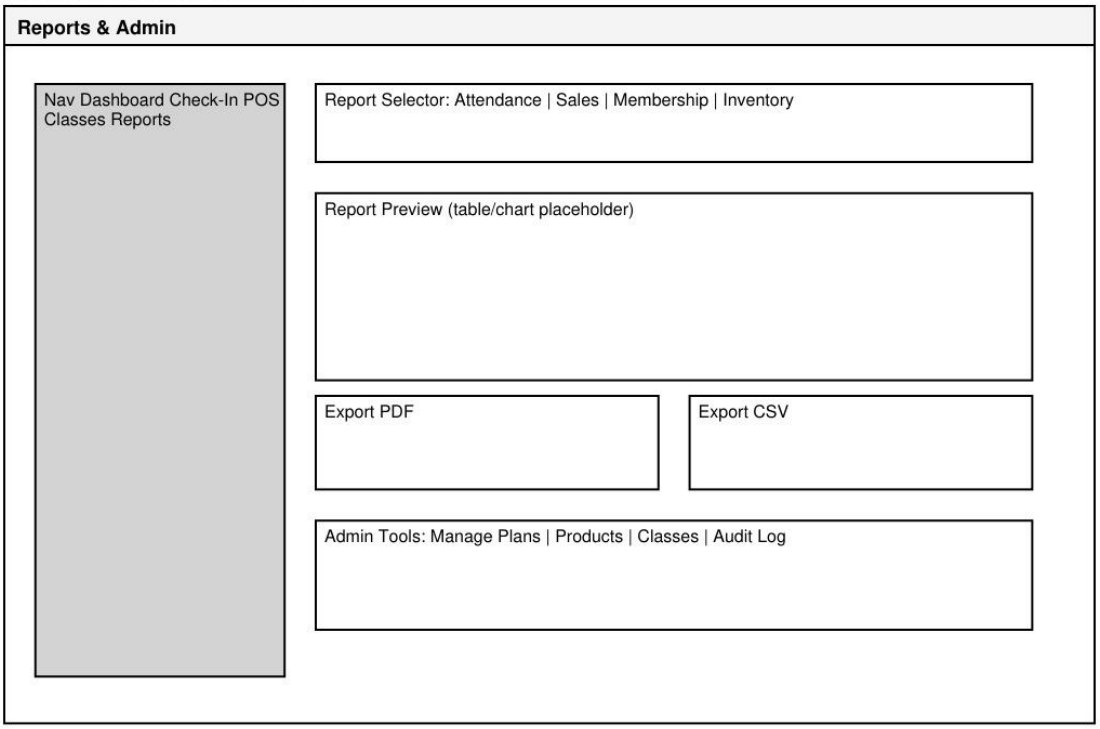



**Figure 9.7: Reports & Admin Screen**

***Screen: Reports & Admin***




## 9.6 Interaction Descriptions

### Login

- User enters username/email and password
- System validates credentials
- Redirects user based on role
- Displays error messages for invalid input

### Dashboard / Home

- Displays quick actions (New Sale, New Member, Check-In, Create Class)
- Summary tiles show key metrics
- Alerts highlight issues such as expired memberships or sync failures

### Member Check-In

- Staff searches member or scans QR code
- System displays membership status and alerts

- Staff confirms check-in
- If offline, data is queued and marked Pending Sync

**Point-of-Sale (POS)**

- Staff searches or selects products
- Items added to cart update total in real time
- Staff selects payment method and completes sale
- System records transaction and provides receipt options

**Classes & Enrollment**

- Staff navigates schedule using filters
- Selects class to view details and capacity
- Enrolls or waitlists members
- System validates eligibility and updates roster

**Reports & Admin**

- Manager selects report type and date range
- System displays report preview
- Reports can be exported (PDF or CSV)
- Admin tools allow updates to plans, products, and classes

•

# 10.0 Program Logic & System Behavior Design

## 10.1 Program Design Overview

Program design defines how GymSync processes user actions and system events using structured logic.

This section translates system requirements, use cases, and process models into step-by-step behavior that determines how the system responds when users:

- Log in
- Check in members
- Process sales
- Enroll members in classes

The logic is derived directly from system workflows and UI interactions. Each feature in the original design includes:

- Triggering actions
- Validation steps
- Decision logic
- System outputs and responses

Backend services such as authentication, membership validation, inventory, payments, and synchronization are represented conceptually and simulated using mock data.

## 10.2 Feature Identification

The GymSync prototype includes the following core features:

| Feature | Description |
|---|---|
| Login & Role-Based Access | Authenticates users and routes based on role |
| Member Check-In | Validates membership and records visits |
| Point-of-Sale (POS) | Processes transactions and payments |
| Class Enrollment | Manages class capacity and enrollment |
| Offline Sync Handling | Queues and synchronizes offline actions |

These features align directly with system use cases and workflows.

## 10.3 Program Logic Descriptions

### 10.3.1 Login & Role-Based Access

**Trigger:**
User enters credentials and selects “Sign In”

**Processing Logic:**

- Validate required fields
- Send authentication request
- Receive response and determine role
- Load appropriate dashboard

**Outputs / System Responses:**

- Success: User is redirected to role-based dashboard
- Failure: Error message displayed and input highlighted
- Security case: Unauthorized roles are logged and blocked

**Edge Cases:**

- Missing credentials
- Invalid login attempts
- Account locked after multiple failures

### 10.3.2 Member Check-In (Online/Offline)

**Trigger:**
Staff selects a member and clicks “Check In”

**Processing Logic:**

- Validate member selection
- Retrieve membership status
- Check for holds or expiration
- Create check-in record

**Outputs / System Responses:**

- Valid membership: Check-in recorded successfully
- Expired membership: Prompt renewal or payment
- Hold present: Block check-in unless overridden

- Offline: Save check-in locally and mark as “Pending Sync”

**Edge Cases:**

- Member not selected
- Duplicate check-in attempts
- Network failure during submission

**Decision Logic Summary:**
As shown in the decision table:

- Active + Online → Save to system
- Active + Offline → Queue locally
- Inactive → Prompt renewal
- Hold → Block action

### 10.3.3 Point-of-Sale (POS) Transaction

**Trigger:**
Staff adds items to cart and selects “Complete Sale”

**Processing Logic:**

- Validate cart is not empty
- Validate payment method
- Calculate totals (subtotal, tax, discounts)
- Create sale and payment records

**Outputs / System Responses:**

- Success: Sale completed and receipt generated
- Failure: Error displayed and transaction halted
- Offline: Transaction saved locally as “Pending Sync”

**Edge Cases:**

- Empty cart
- Missing payment method
- Payment mismatch
- Network interruption

### 10.3.4 Class Enrollment & Capacity Management

**Trigger:**

Staff selects class and chooses “Enroll” or “Waitlist”

**Processing Logic:**

- Validate member and class selection
- Check membership eligibility
- Check class capacity
- Assign enrollment status

**Outputs / System Responses:**

- Enrolled: Member added to roster
- Full class: Member added to waitlist
- Ineligible: Enrollment blocked with message
- Offline: Enrollment queued for sync

**Edge Cases:**

- Duplicate enrollment
- Class capacity changes
- Membership restrictions

### 10.4 Error Handling & Validation Logic

GymSync validates user input at two levels:

**Front-End Validation**

- Required fields (login, member selection, cart items)
- Data format validation (phone numbers, dates, payment amounts)

**Conceptual Backend Validation**

- Membership must be active
- Holds block check-in and enrollment
- Payment must match total
- Duplicate records are prevented

**System Feedback**

- Inline error messages for minor issues
- Modal alerts for critical errors
- Success confirmations via notifications or status messages

## 10.5 Offline Handling & System Behavior

GymSync supports offline operation by:

- Storing actions in a local queue
- Marking transactions as “Pending Sync”
- Displaying offline indicators to the user

When connectivity is restored:

- The system replays queued actions
- Updates each action as “Synced” or “Failed”
- Prompts user review for conflicts

## 10.6 Backend Simulation Strategy

The prototype simulates backend functionality using Windsurf.

**Simulation Approach**

- Mock datasets for members, products, classes, and transactions
- Service functions simulate API responses
- UI reflects real system behavior

**Simulation Features**

- Authentication responses
- Membership validation
- Payment processing results
- Sync status indicators

**Offline Simulation**

- UI toggle for Online/Offline mode
- Queue processing on reconnect
- Conflict handling prompts

As described in the prototype strategy, the system maintains consistent behavior even without a real backend.

# 11.0 Data Storage & State Management Design

## 11.1 Data Storage Overview

Data storage defines where GymSync data resides, how it is accessed, and how it persists to support system operations.

GymSync supports core processes such as:

- Member check-in
- Point-of-sale (POS) transactions
- Class enrollment
- Offline synchronization

Conceptual backend storage represents how a complete system would store data using relational structures, including tables, keys, and relationships.

The prototype simulates this behavior using mock data and front-end state management instead of a real database.

## 11.2 Data Storage Requirements

GymSync manages two categories of data:

**Persistent Data (Long-Term Storage)**

- Member profiles
- Sales transactions
- Class sessions and enrollments
- Products and inventory
- Payment records
- Notifications and audit logs

**Temporary Data (Session / UI State)**

- Logged-in user context
- Selected member
- Shopping cart contents
- Active screen and workflow state
- Offline queue status

In the data table, each data type supports specific system features and use cases.

## 11.3 Conceptual Storage Design

GymSync uses a **relational data model (conceptual)** organized into logical groups:

**Identity & Access**

- StaffUser
- Role
- Permissions

**Membership**

- Member
- MembershipPlan
- MemberMembership

**Operations**

- CheckIn
- ClassSession
- Enrollment

**Commerce**

- Product
- Sale
- SaleLineItem
- PaymentTransaction

**Communication**

- Notification

**Governance**

- AuditLog / HistoryArchive

## 11.4 Key Data Relationships (Expanded)

To strengthen structural clarity, the following relationships define how data entities connect:

- **Member → Enrollment (1:M)**

A member can enroll in multiple classes through Enrollment records.

- **ClassSession → Enrollment (1:M)**
  A class session contains multiple enrollments.
- **Member ↔ ClassSession (M:N)**
  Resolved through the Enrollment entity.

- **Sale → SaleLineItem (1:M)**
  Each sale contains multiple line items representing products purchased.
- **Product → SaleLineItem (1:M)**
  Each line item references a product.
- **Sale → PaymentTransaction (1:M)**
  A sale may include multiple payments (e.g., split payments).

- **Member → CheckIn (1:M)**
  Each member can have multiple check-in records.
- **OfflineSyncQueue → System Actions (1:M)**
  Stores queued actions such as check-ins, sales, and enrollments for later synchronization.

These relationships ensure data consistency, normalization, and accurate representation of system behavior.

### 11.5 Data Lifecycle & State Management

GymSync data follows a structured lifecycle:

**Creation**

- CheckIn created during member visit
- Sale, SaleLineItem, and PaymentTransaction created during POS transaction
- Enrollment created during class registration

**Modification**

- Membership status updates (active, expired, hold)
- Enrollment updates (enrolled, waitlisted, cancelled)
- Inventory updates after sales

**Retrieval**

- Member lookup during check-in

- Product retrieval during POS
- Class schedule retrieval during enrollment
- Reports and audit data accessed by managers

**Archival**

- Historical records retained for reporting and compliance
- Soft deletes used instead of permanent removal

### 11.6 State Management During User Interaction (Expanded)

During system operation, GymSync maintains a **temporary UI state** that reflects current user activity.

This includes:

- Logged-in user and role
- Selected member context
- Cart contents and totals
- Current class schedule and roster view
- Network status (Online / Offline)
- Pending Sync queue

### State Synchronization Behavior

State changes occur as users interact with the system and must remain consistent with stored prototype data.

**Example Flow: Member Check-In**

1. Staff selects a member → UI state updates with selected member
2. Staff clicks "Check-In" → temporary state records action
3. If online:
    a. Action is processed immediately
    b. Persistent data is updated
4. If offline:
    a. Action is stored in OfflineSyncQueue
    b. UI displays "Pending Sync" status

**Example Flow: POS Transaction**

1. Items added to cart → state updates cart totals

2. Payment selected → state tracks payment method
3. On “Complete Sale”:
    a. Online: Save to persistent dataset
    b. Offline: Save to queue and mark pending

**State Consistency Across Navigation**

- State persists across screens using in-memory variables
- Key data (cart, selected member, queue) is preserved during navigation
- UI updates reflect latest state after each action

**Synchronization with Stored Data**
When transitioning from offline to online:

- Queued actions are replayed
- Each action updates persistent data
- UI state is refreshed to reflect synced results
- Failed actions trigger user review

## 11.7 Prototype Data Simulation Strategy

The GymSync prototype simulates backend behavior using Windsurf.

**Simulation Techniques**

- Static mock datasets (members, products, classes, staff)
- Front-end state variables for session context
- Local storage (localStorage / IndexedDB) for offline queue persistence
- Predefined service functions for simulated API responses

**Offline Simulation**

- Network toggle allows switching between Online and Offline modes
- Actions are queued while offline
- On reconnect:
    - Actions replay in order
    - Marked as “Synced” or “Failed”
    - Summary displayed to user

**Limitations**

- No real multi-user concurrency
- No real payment processing
- Conflict handling is simulated
- Performance optimizations (indexing, scaling) are conceptual

## 12.0 Product Requirements Document, Prototype Implementation & System Demonstration

### 12.1 PRD Overview

The Product Requirements Document (PRD) for GymSync defines the system behavior, user interactions, and implementation expectations required to build a functional prototype.

GymSync is a gym management information system designed to support:

- Member management
- Check-in operations
- Class scheduling and enrollment
- Point-of-sale (POS) transactions
- Notifications and system feedback

The full system is conceptually designed with backend services, APIs, and persistent storage. However, this project implements a front-end-only prototype that simulates backend behavior using mock data, temporary UI state, and browser storage.

This PRD is structured to guide Windsurf (AI) in generating a working prototype aligned with the system design.

### 12.2 System Goals and Users

#### System Goals

- Provide a unified system for gym operations (check-in, booking, sales)
- Reduce manual work and improve workflow efficiency
- Demonstrate real system behavior through a working prototype
- Translate SDLC design artifacts into an executable PRD
- Show realistic state changes and user feedback

#### Intended Users

| User Role | Responsibilities |
|---|---|
| Front Desk Staff | Check in members, process sales, manage records |

| | |
|---|---|
| Managers | Review reports, monitor activity, oversee operations |
| Members | View profiles, book classes, receive notifications |

### 12.3 Functional Requirements Traceability

The prototype directly implements key functional requirements:

| Requirement | Implementation |
|---|---|
| FR-1, FR-2 | Member Check-In screen (search, validation, confirmation) |
| FR-3, FR-4 | Class Booking screen (view sessions, enroll, capacity updates) |
| FR-5, FR-6 | POS screen (cart, totals, payment simulation) |
| FR-7 | Member profile view/edit |
| FR-8 | Dashboard summaries |
| FR-9 | Notifications and alerts |
| FR-10 | State persistence using mock data and localStorage |

This ensures alignment between the PRD and the working prototype.

### 12.4 Assumptions and Constraints

#### Assumptions

- The system is designed as a full client-server application conceptually
- Users interact through a browser-based interface
- Sample data is sufficient to simulate real workflows
- Staff follow defined operational processes
- Offline behavior is expected and must be represented

#### Constraints

- The prototype is limited to front-end implementation only
- No real API or database is used
- Data persistence is simulated using JavaScript objects and localStorage
- Payment processing is simulated (no real transactions)
- Multi-user concurrency is not supported
- Development is constrained to project scope and timeline

## 12.5 Prototype Implementation Using Windsurf

The GymSync prototype was developed using Windsurf as the primary AI coding

assistant.

### Development Approach

- PRD used as structured input for Windsurf
- Iterative prompting refined UI and behavior
- Features built incrementally based on system workflows

### Technology Stack

- HTML, CSS, JavaScript
- Mock data (JSON objects)
- localStorage for persistence simulation

### Core Features Implemented

- Dashboard with system summaries
- Member check-in workflow
- Class booking system
- POS transaction interface
- Notifications and alerts
- Offline simulation with status indicators

## 12.6 System Behavior & Workflow Execution

### Member Check-In

- Staff selects or searches member
- System validates membership status
- Active → check-in recorded and dashboard updated
- Inactive → warning displayed

### Class Booking

- User selects class session
- System checks capacity
- Available → enrollment created, seats decrease
- Full → error or waitlist message

### Point-of-Sale (POS)

- Items added to cart
- System calculates subtotal, tax, total
- Mock payment processed
- Sale recorded and receipt displayed

### Offline Simulation

- Offline mode toggle available
- Actions stored as **"Pending Sync"**
- When online:
  - Actions marked as **"Synced"**
  - UI updates accordingly

## 12.7 Data Simulation Strategy

The prototype simulates backend storage using front-end structures:

| Entity | Purpose | Prototype Representation |
|---|---|---|
| Member | Profile and status | Mock data + localStorage |
| ClassSession | Schedule and capacity | Dynamic array |
| Enrollment | Member-class relationship | State-managed array |
| Product | POS items | Static dataset |
| Sale | Transaction record | Generated in state |
| SaleLineItem | Items in sale | Derived from cart |
| Notification | Alerts/messages | Generated dynamically |

This structure maintains consistency with the conceptual data model.

## 12.8 PRD Instructions for AI Coding Assistant

### Implementation Guidelines

- Build a multi-screen web application
- Use consistent navigation and layout
- Implement:
  - Member check-in
  - Class booking
  - POS transactions
- Use mock data and localStorage

- Simulate backend responses
- Display clear status indicators and feedback

### Build Order

1. Create application layout and navigation
2. Build member check-in feature
3. Build class booking feature
4. Build POS feature
5. Add notifications and offline simulation
6. Refine UI and test workflows

## AI Prompt Starter

Create a front-end-only web prototype for a gym management system called "GymSync" using HTML, CSS, and JavaScript. Include a dashboard, check-in screen, class booking system, and POS interface. Use mock data and localStorage to simulate persistence. Implement working features such as check-in updates, class booking capacity changes, POS calculations, and an offline mode with "Pending Sync" status indicators.

## 12.9 Testing and Validation

The prototype was tested using:

- **Unit Testing:** calculations and state updates
- **Integration Testing:** feature interactions across screens
- **System Testing:** full workflow execution
- **Usability Testing:** ease of use for staff workflows
- **Acceptance Testing:** alignment with requirements
- **Documentation Testing:** PRD matches system behavior

### Sample Test Cases

| ID | Scenario | Expected Outcome |
|---|---|---|
| TC-1 | Check in active member | Success message and dashboard update |
| TC-2 | Book full class | Error or waitlist message |
| TC-3 | Add products to cart | Correct totals calculated |
| TC-4 | Complete sale | Receipt shown and cart cleared |
| TC-5 | Offline action | Marked "Pending Sync" |

## 12.10 System Demonstration & Deployment

The GymSync prototype was demonstrated through:

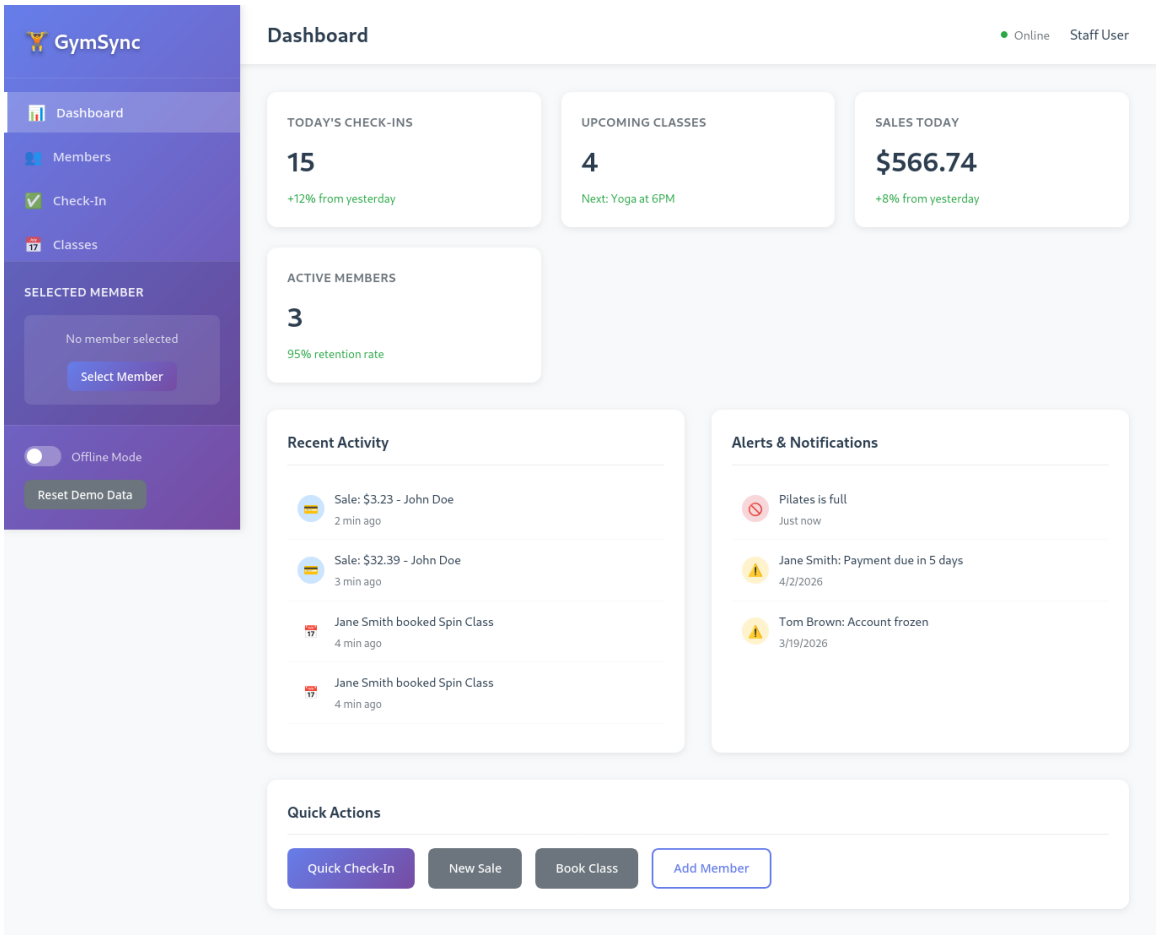
- A recorded walkthrough showing:
  - PRD structure
  - Code implementation
  - Feature execution
- A published version of the system
- A GitHub repository containing:
  - Source code
  - Project structure
  - Implementation files
  - Video and text file containing a YouTube link.
  - Live Demo

This confirms that the PRD, system design, and prototype are fully aligned.

## 12.11 Live Demo and GitHub Links

- **Live Demo:** https://webdevkeenan.github.io/gymsync/
- **Back Up Demo:** https://gymsync26.netlify.app/
- **GitHub:** https://github.com/webdevkeenan/gymsync

*Figure 12.1*




***Description:***
The image above is a screenshot of the live demo of **GymSync**, showcasing the platform interface which is designed to accurately record data in the event of the lack of online connectivity.

## 12.12 Implementation Summary

The GymSync prototype successfully demonstrates:

- Translation of system requirements into working features
- Integration of UI, logic, and simulated data
- Realistic workflows with visible state changes
- Use of AI-assisted project development (Windsurf)

This section represents the final stage of the SDLC, connecting system design to a functional implementation.

# 13.0 Conclusion

## 13.1 Project Summary

The GymSync system demonstrates a complete system design developed using the Systems Development Life Cycle (SDLC). The project integrates requirements analysis, process modeling, data design, architecture planning, and user interface development into a cohesive solution.

## 13.2 Key Outcomes

This project successfully:

- Defined system requirements and user interactions
- Modeled workflows and data relationships
- Designed a scalable, offline-capable system architecture
- Developed user interface wireframes aligned with real workflows
- Created program logic and state management strategies for system behavior

## 13.3 Real-World Application

GymSync reflects real-world business needs by addressing challenges such as unreliable internet connectivity, fast-paced front-desk operations, and secure transaction processing. The system design can be extended into a full implementation in a production environment.

## 13.4 Closing

Through this capstone, the full SDLC process was applied to design a realistic and functional system. The project demonstrates the ability to translate business requirements into a structured technical solution and prepares future work in system analysis, design, and development.